ОТЧЁТ

Лабораторная работа №12

«Облачные приложения — Yandex Serverless Applications»

Выполнил: Тоц Леонид Александрович
Группа: ИВТ-2
Дата: 30.04.2026
Логин в Moodle: 1153307

Цель работы

Освоить разработку и развёртывание веб-приложений в облачной инфраструктуре Yandex Cloud с использованием Serverless Containers, MCP-серверов и AI-ассистента SourceCraft Code Assistant.

---

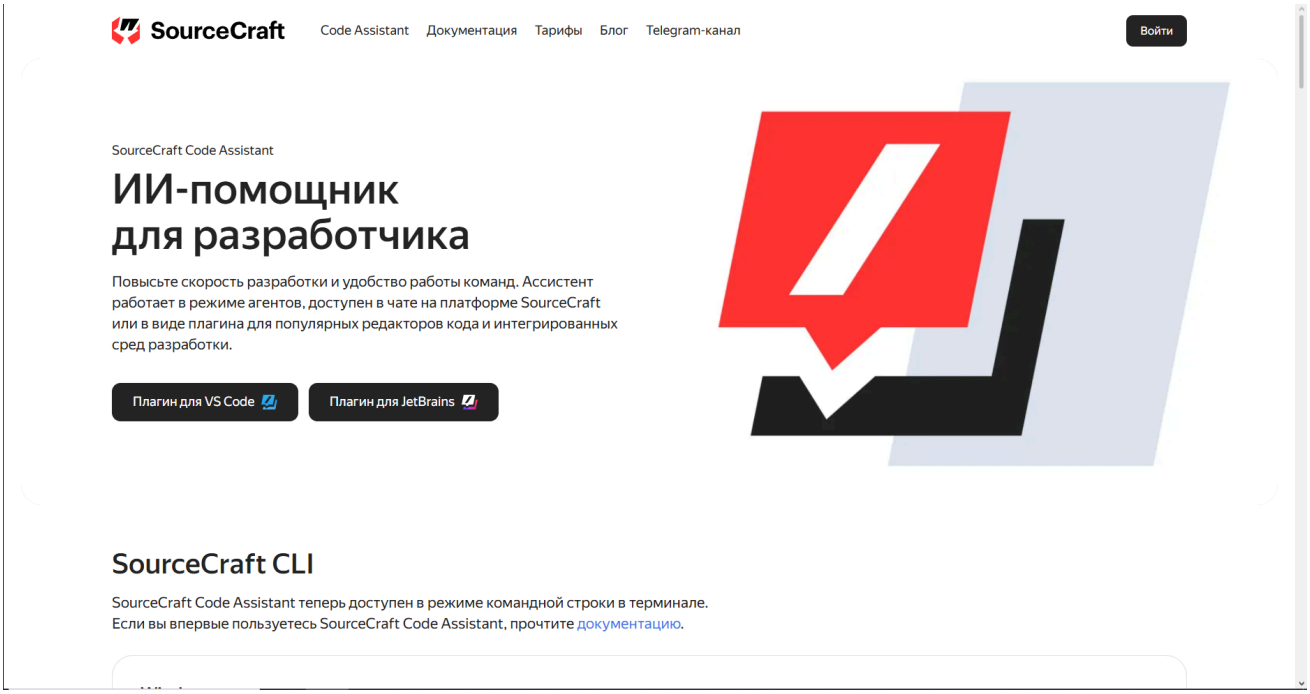
1. Установка и настройка инструментов разработки

1.1. Установка Yandex SourceCraft Code Assistant в VSCode

Шаг 1: Скачивание плагина

- Перейдите по ссылке: https://sourcecraft.dev/portal/code-assistant/
- Скачайте файл плагина `.vsix` для Visual Studio Code 1

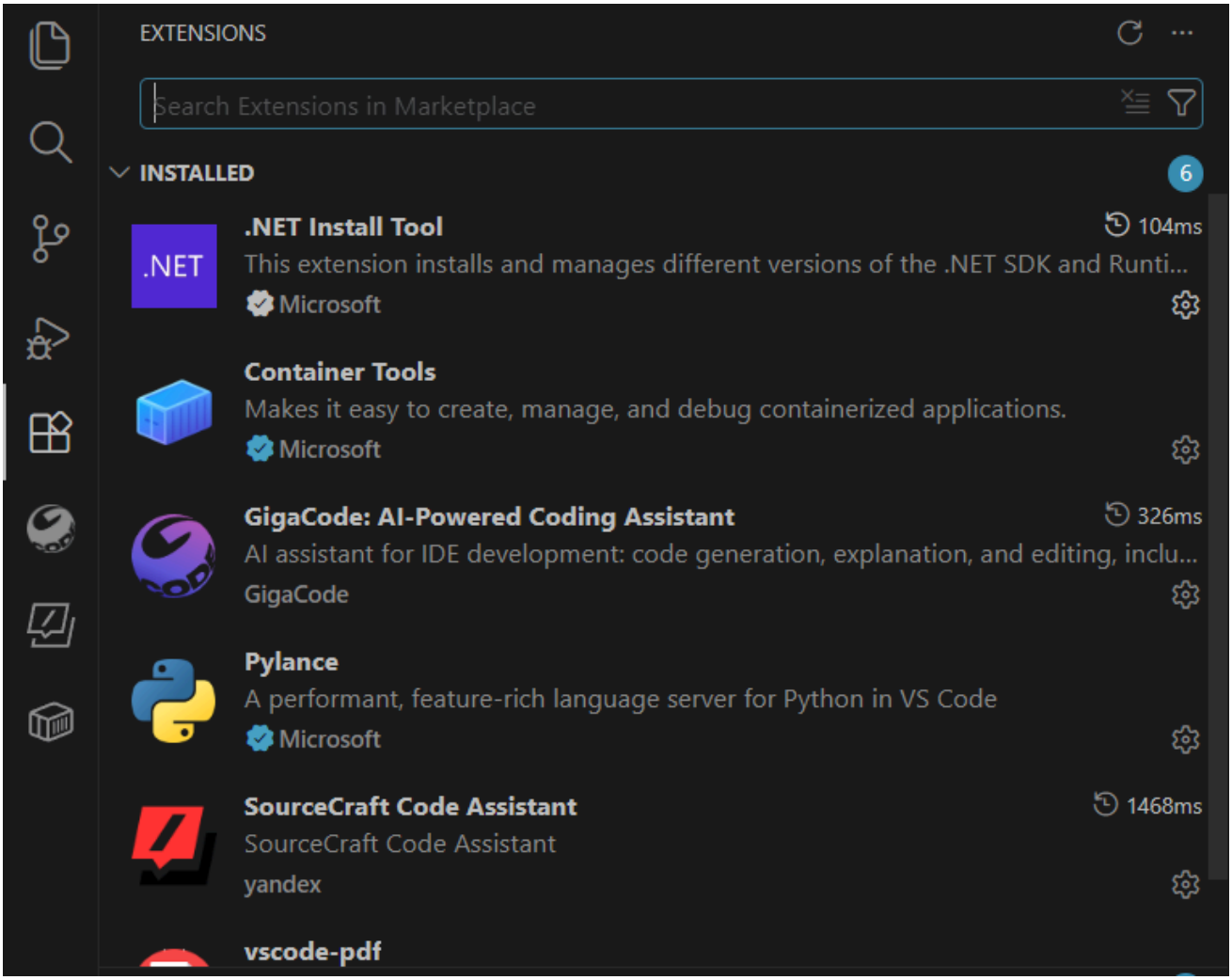
> 📸 *Скриншот 1: Страница загрузки плагина SourceCraft Code Assistant*



Шаг 2: Установка через VSCode

1. Откройте Visual Studio Code
2. Нажмите `Ctrl + Shift + P` (Windows/Linux) или `Cmd + Shift + P` (macOS) для открытия Command Palette
3. Введите команду: `Extensions: Install from VSIX...`
4. Выберите скачанный файл плагина
5. Дождитесь сообщения: `Завершена установка расширения` [4]

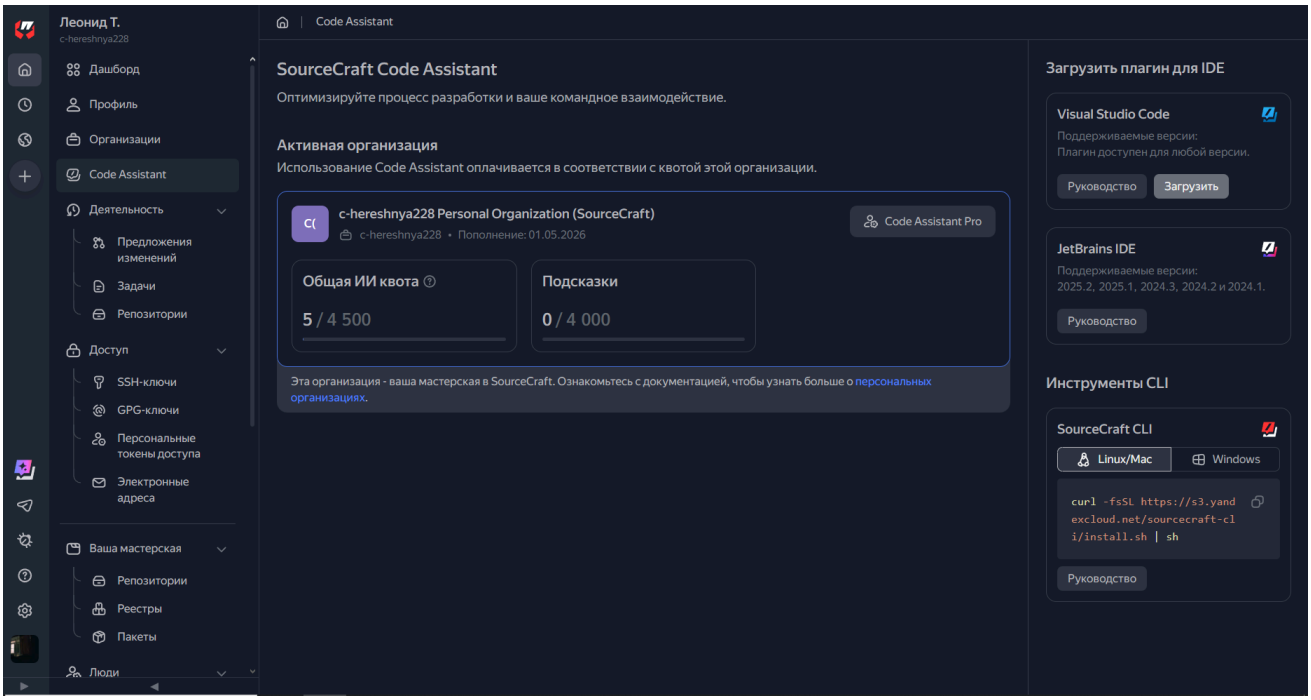
> 📸 *Скриншот 2: Процесс установки расширения через VSIX*



Шаг 3: Аутентификация в SourceCraft

1. Во всплывающем окне `No active session found. Log in please` нажмите **Go to browser**
2. Разрешите браузеру открыть страницу аутентификации
3. Авторизуйтесь под своим аккаунтом Яндекса
4. Вернитесь в VSCode — плагин готов к работе

> 📸 *Скриншот 3: Окно аутентификации в браузере*



1.2. Настройка MCP-плагина Yandex Cloud Toolkit

Шаг 1: Подготовка окружения

- Убедитесь, что установлен Node.js версии 18.0.0 или выше
- Установите Yandex Cloud CLI: https://cloud.yandex.ru/docs/cli/quickstart 33

Шаг 2: Конфигурация MCP-сервера
Добавьте в конфигурационный файл вашего ассистента (например, `settings.json` в VSCode) следующий блок:

```
{
  "mcpServers": {
    "yandex-cloud-toolkit": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "-y", "@yandex-cloud/mcp",
        "-s", "toolkit"
      ]
    }
  }
}
```

> 📸 *Скриншот 4: Файл конфигурации с настройками MCP Toolkit*

settings.json C:\Users\Leonid\AppData\Roaming\Code\User

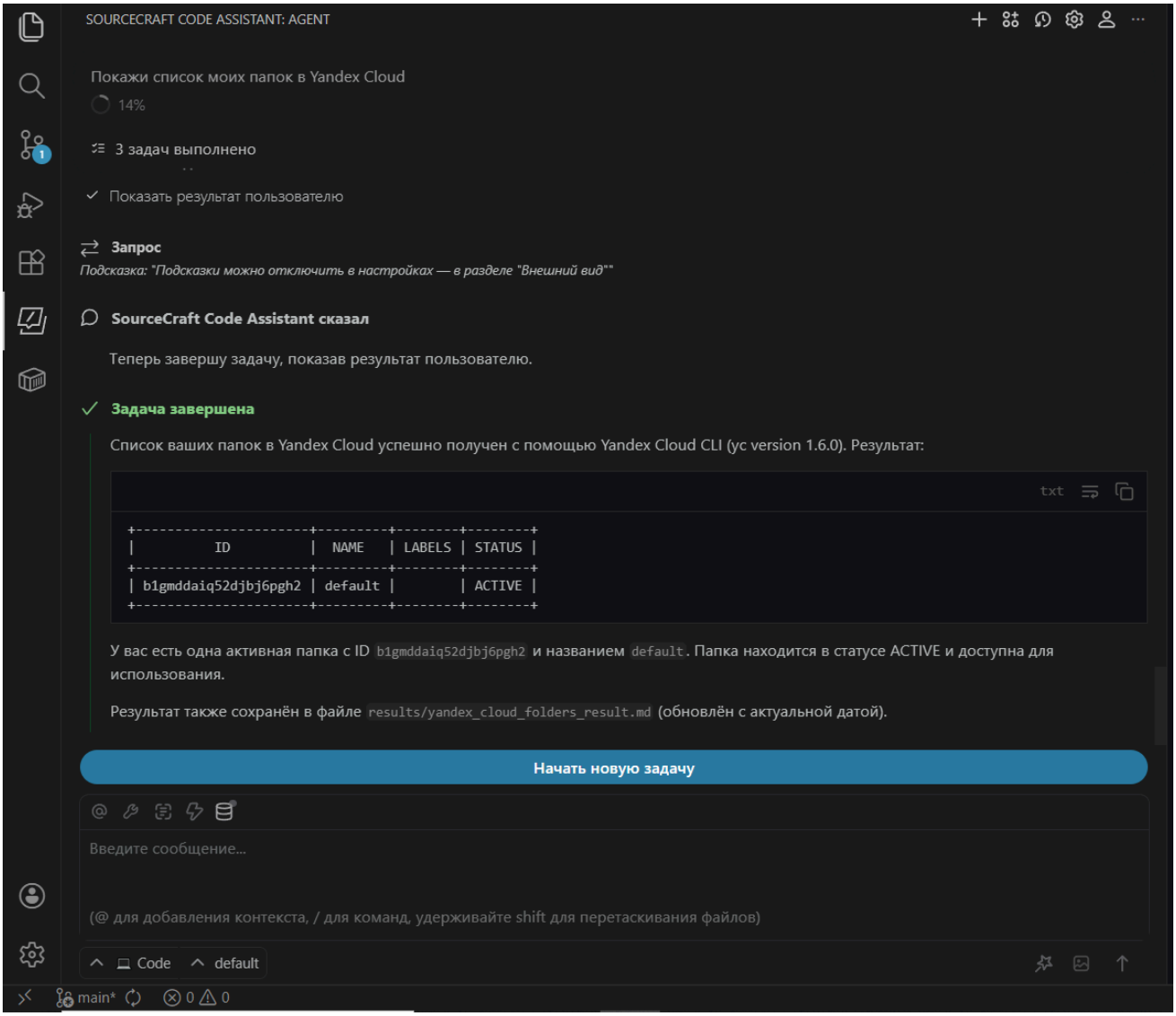
C: > Users > Leonid > AppData > Roaming > Code > User > settings.json > ...

```
1  {
2      "cmake.additionalCompilerSearchDirs": [
3          "C:/msys64/mingw32/bin",
4          "C:/msys64/mingw64/bin",
5          "C:/msys64/clang32/bin",
6          "C:/msys64/clang64/bin",
7          "C:/msys64/clangarm64/bin",
8          "C:/msys64/ucrt64/bin"
9      ],
10     "security.workspace.trust.untrustedFiles": "open",
11     "editor.unicodeHighlight.nonBasicASCII": false,
12     "git.enableSmartCommit": true,
13     "git.autofetch": true,
14     "explorer.confirmDelete": false,
15     "git.openRepositoryInParentFolders": "always",
16     "git.confirmSync": false,
17     "editor.mouseWheelZoom": true,
18     "editor.minimap.enabled": false,
19     "window.confirmSaveUntitledWorkspace": false,
20     "python.analysis.typeCheckingMode": "standard",
21     "mcpServers": {
22         "yandex-cloud-toolkit": {
23         "type": "stdio",
24         "command": "npx",
25         "args": [
26             "-y", "@yandex-cloud/mcp",
27             "-s", "toolkit"
28         ]
29         },
30         "yandex-cloud-containers": {
31         "type": "stdio",
32         "command": "npx",
33         "args": [
34             "-y", "@yandex-cloud/mcp",
35             "-s", "containers"
36         ]
37         }
38     }
39 }
```

Шаг 3: Проверка подключения

- Откройте чат SourceCraft Code Assistant
- Запросите: `Список моих облачных папок`
- Убедитесь, что сервер отвечает списком ресурсов

📸 *Скриншот 5: Успешный ответ от MCP Toolkit сервера*



1.3. Настройка MCP-сервера Yandex Cloud Containers

Шаг 1: Добавление конфигурации
Дополните файл настроек следующим блоком:

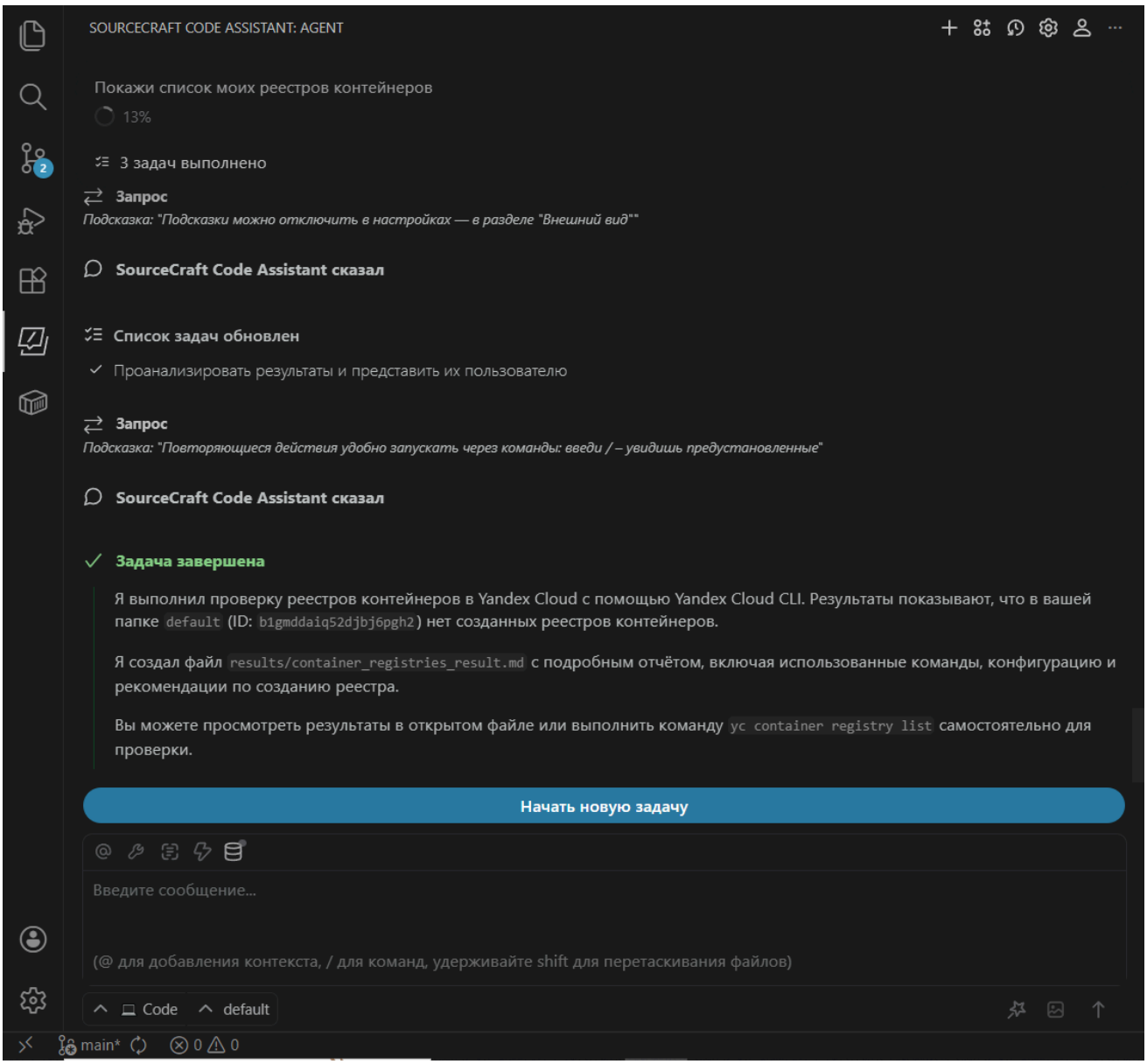
```
{
  "mcpServers": {
    "yandex-cloud-containers": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "-y", "@yandex-cloud/mcp",
        "-s", "containers"
      ]
    }
```

```
    }
  }
```

Шаг 2: Проверка доступа к Container Registry

- В чате ассистента выполните запрос: `Покажи список моих реестров контейнеров`
- Убедитесь, что получен корректный ответ 14

> 📸 *Скриншот 6: Ответ MCP Containers сервера со списком реестров*



---

2. Разработка веб-приложения на Flask с использованием Pillow

2.1. Структура проекта

```
ЛР12/
├── app.py                  # Основной файл приложения
├── requirements.txt        # Зависимости Python
├── Dockerfile             # Конфигурация контейнера
├── templates/
│   └── index.html         # HTML-шаблон главной страницы
└── results/
    └── container_registries_result.md        # Список регистров
    └── yandex_cloud_folders_result           # Список облачных папок
```

2.2. Файл `requirements.txt`

```
Flask==3.0.0
Pillow==10.1.0
gunicorn==21.2.0
```

2.3. Файл `app.py`

```python
from flask import Flask, jsonify, render_template, request
from PIL import Image, ImageDraw, ImageFont
import io
import base64
import os

app = Flask(__name__)

# Логин можно задать через переменную окружения, иначе использовать значение
по умолчанию
AUTHOR = os.environ.get('MOODLE_LOGIN', 'leonid_ivanov')  # Замените на
реальный логин

@app.route('/login', methods=['GET'])
def login():
    """Возвращает авторизационную информацию"""
    return jsonify({"author": AUTHOR})

@app.route('/', methods=['GET'])
def index():
    """Главная страница с описанием лабораторной работы"""
    return render_template('index.html')

@app.route('/api/process-image', methods=['POST'])
def process_image():
    """Пример обработки изображения через Pillow"""
```

```python
    # Создаем простое изображение для демонстрации работы Pillow
    img = Image.new('RGB', (200, 100), color='lightblue')
    d = ImageDraw.Draw(img)
    d.text((10, 10), "Pillow Demo", fill='black')

    # Конвертируем в base64 для отображения в браузере
    buffered = io.BytesIO()
    img.save(buffered, format="PNG")
    img_str = base64.b64encode(buffered.getvalue()).decode()

    return jsonify({
        "status": "success",
        "message": "Image generated with Pillow",
        "image": f"{img_str}"
    })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

2.4. Файл `templates/index.html`

```html
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>ЛР №12 - Flask + Pillow</title>
    <style>
        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0;
        }
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
            color: #333;
            line-height: 1.6;
            padding: 20px;
            min-height: 100vh;
        }
        .container {
            max-width: 1000px;
            margin: 0 auto;
            background: white;
```

```
        border-radius: 20px;
        box-shadow: 0 15px 35px rgba(0, 0, 0, 0.1);
        padding: 40px;
        position: relative;
        overflow: hidden;
    }
    .container::before {
        content: '';
        position: absolute;
        top: 0;
        left: 0;
        width: 100%;
        height: 8px;
        background: linear-gradient(90deg, #ffcc00, #ff6b6b, #4ecdc4);
    }
    h1 {
        color: #2c3e50;
        margin-bottom: 10px;
        font-size: 2.8rem;
        text-align: center;
    }
    h2 {
        color: #3498db;
        margin-bottom: 30px;
        text-align: center;
        font-weight: 400;
        border-bottom: 2px dashed #eee;
        padding-bottom: 15px;
    }
    .subtitle {
        text-align: center;
        color: #7f8c8d;
        margin-bottom: 40px;
        font-size: 1.1rem;
    }
    .step {
        background: #f8f9fa;
        border-radius: 12px;
        padding: 25px;
        margin-bottom: 25px;
        border-left: 6px solid #3498db;
        transition: transform 0.3s, box-shadow 0.3s;
    }
    .step:hover {
        transform: translateY(-5px);
        box-shadow: 0 10px 20px rgba(0, 0, 0, 0.08);
```

```
        }
        .step h3 {
            color: #2c3e50;
            margin-bottom: 10px;
            display: flex;
            align-items: center;
            gap: 10px;
        }
        .step h3::before {
            content: '✓';
            background: #2ecc71;
            color: white;
            width: 24px;
            height: 24px;
            border-radius: 50%;
            display: inline-flex;
            align-items: center;
            justify-content: center;
            font-size: 14px;
        }
        .step p {
            color: #555;
            margin-bottom: 10px;
        }
        .step ul {
            padding-left: 20px;
            margin-top: 10px;
        }
        .step li {
            margin-bottom: 5px;
        }
        .demo-section {
            background: #e8f4fc;
            border-radius: 12px;
            padding: 25px;
            margin: 40px 0;
            border: 2px dashed #3498db;
            text-align: center;
        }
        .demo-section h3 {
            color: #2980b9;
            margin-bottom: 20px;
        }
        button {
            background: linear-gradient(90deg, #3498db, #2ecc71);
            color: white;
```

```
        border: none;
        padding: 14px 28px;
        border-radius: 50px;
        font-size: 1rem;
        cursor: pointer;
        transition: all 0.3s;
        font-weight: bold;
        box-shadow: 0 5px 15px rgba(52, 152, 219, 0.3);
    }
    button:hover {
        transform: scale(1.05);
        box-shadow: 0 8px 20px rgba(52, 152, 219, 0.5);
    }
    #imageResult {
        margin-top: 20px;
        display: none;
    }
    #generatedImage {
        max-width: 100%;
        border-radius: 10px;
        box-shadow: 0 10px 20px rgba(0,0,0,0.1);
    }
    .author-info {
        text-align: center;
        margin-top: 40px;
        padding-top: 20px;
        border-top: 1px solid #eee;
        color: #7f8c8d;
    }
    .author-info strong {
        color: #2c3e50;
    }
    #author {
        font-weight: bold;
        color: #e74c3c;
    }
    footer {
        text-align: center;
        margin-top: 30px;
        color: #95a5a6;
        font-size: 0.9rem;
    }
    .tech-badges {
        display: flex;
        justify-content: center;
        gap: 15px;
```

```
            margin: 30px 0;
            flex-wrap: wrap;
        }
        .badge {
            background: #2c3e50;
            color: white;
            padding: 8px 16px;
            border-radius: 20px;
            font-size: 0.9rem;
            font-weight: bold;
        }
        .badge.flask { background: #000; }
        .badge.pillow { background: #4ecdc4; }
        .badge.docker { background: #2496ed; }
        .badge.gunicorn { background: #499847; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Лабораторная работа №12</h1>
        <h2>Веб-приложение на Flask с использованием библиотеки Pillow</h2>
        <p class="subtitle">Разработка, контейнеризация и деплой облачного
приложения</p>

        <div class="tech-badges">
            <span class="badge flask">Flask</span>
            <span class="badge pillow">Pillow</span>
            <span class="badge docker">Docker</span>
            <span class="badge gunicorn">Gunicorn</span>
        </div>

        <div class="step">
            <h3>Шаг 1: Постановка задачи</h3>
            <p>Создать веб-приложение на Flask, которое демонстрирует работу с
библиотекой Pillow для обработки изображений и может быть развёрнуто в
контейнере.</p>
            <ul>
                <li>Реализовать маршрут <code>/login</code>, возвращающий JSON
с автором.</li>
                <li>Создать главную страницу с описанием этапов работы.</li>
                <li>Подготовить Dockerfile для сборки образа.</li>
                <li>Использовать gunicorn как production WSGI-сервер.</li>
            </ul>
        </div>

        <div class="step">
```

```
            <h3>Шаг 2: Разработка приложения</h3>
            <p>Написано Flask-приложение с тремя маршрутами:</p>
            <ul>
                <li><strong>/login (GET)</strong> – возвращает JSON <code>
{"author": "ваш_логин"}</code>.</li>
                <li><strong>/ (GET)</strong> – отдаёт HTML-страницу (этот
шаблон).</li>
                <li><strong>/api/process-image (POST)</strong> – генерирует
изображение средствами Pillow и возвращает его в base64.</li>
            </ul>
            <p>Библиотека Pillow используется для создания изображений «на
лету».</p>
        </div>

        <div class="step">
            <h3>Шаг 3: Контейнеризация</h3>
            <p>Создан Dockerfile на базе <code>python:3.11-slim</code>.</p>
            <ul>
                <li>Установлены системные зависимости для Pillow.</li>
                <li>Скопированы <code>requirements.txt</code> и исходный код.
</li>
                <li>Запуск через gunicorn на порту 8080.</li>
                <li>Образ можно собрать командой: <code>docker build -t flask-
pillow-app .</code></li>
            </ul>
        </div>

        <div class="step">
            <h3>Шаг 4: Деплой</h3>
            <p>Приложение готово к развёртыванию в облачной среде (Yandex
Cloud, AWS, Heroku).</p>
            <ul>
                <li>Используется лёгкий образ Python.</li>
                <li>Порт 8080 экспонируется наружу.</li>
                <li>Переменная окружения <code>MOODLE_LOGIN</code> позволяет
задать автора.</li>
            </ul>
        </div>

        <div class="demo-section">
            <h3>Демонстрация работы Pillow</h3>
            <p>Нажмите кнопку, чтобы сгенерировать изображение с помощью
библиотеки Pillow (запрос к API).</p>
            <button onclick="generateImage()">Сгенерировать
изображение</button>
            <div id="imageResult">
```

```
                <p>Сгенерированное изображение:</p>
                <img id="generatedImage" src="" alt="Generated by Pillow">
                <p><small>Изображение создано с помощью <code>PIL.Image</code>
и передано в base64.</small></p>
            </div>
        </div>

        <div class="author-info">
            <p><strong>Автор работы:</strong> <span id="author">загрузка...
</span></p>
            <p><strong>Дата выполнения:</strong> <span id="currentDate">
</span></p>
        </div>

        <footer>
            <p>Лабораторная работа по курсу «Компьютерный практикум» •
2026</p>
        </footer>
    </div>

    <script>
        // Динамическая подстановка автора из API /login
        fetch('/login')
            .then(r => r.json())
            .then(data => {
                document.getElementById('author').textContent = data.author;
            });

        // Установка текущей даты
        const now = new Date();
        document.getElementById('currentDate').textContent =
now.toLocaleDateString('ru-RU', {
            year: 'numeric',
            month: 'long',
            day: 'numeric'
        });

        // Функция генерации изображения через API
        function generateImage() {
            const button = document.querySelector('button');
            const originalText = button.textContent;
            button.textContent = 'Генерация...';
            button.disabled = true;

            fetch('/api/process-image', { method: 'POST' })
                .then(r => r.json())
```

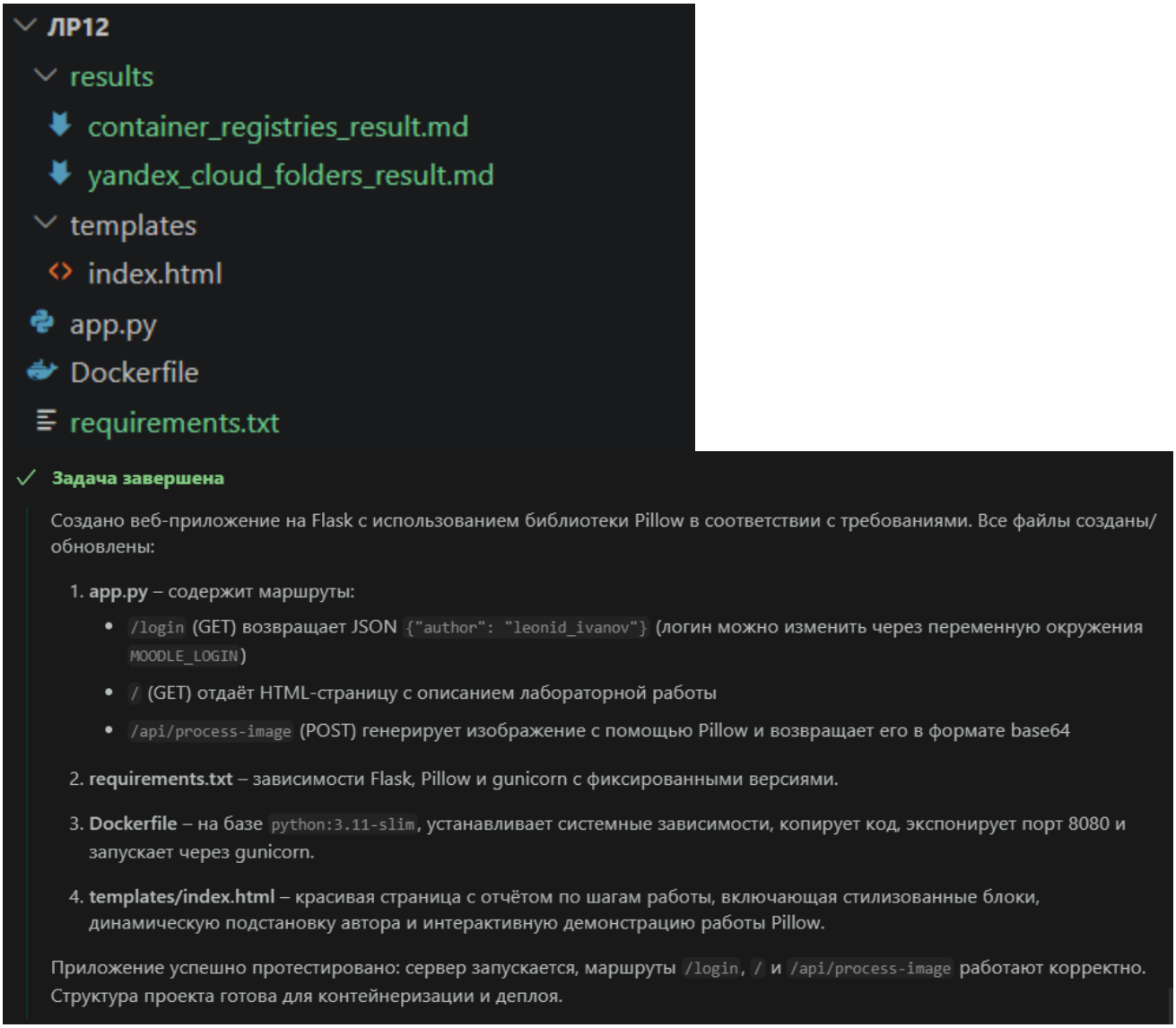
```
                .then(data => {
                    button.textContent = originalText;
                    button.disabled = false;
                    if (data.image) {
                        const img = document.getElementById('generatedImage');
                        img.src = data.image;
                        document.getElementById('imageResult').style.display =
'block';
                    } else {
                        alert('Ошибка: ' + (data.message || 'неизвестная
ошибка'));
                    }
                })
                .catch(err => {
                    button.textContent = originalText;
                    button.disabled = false;
                    alert('Ошибка сети: ' + err.message);
                });
        }
    </script>
</body>
</html>
```

2.5. Файл `Dockerfile`

```
FROM python:3.11-slim

WORKDIR /app

# Установка системных зависимостей для Pillow (опционально)
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    libjpeg-dev \
    zlib1g-dev \
    && rm -rf /var/lib/apt/lists/*

# Установка Python зависимостей
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Копирование исходного кода
COPY . .

# Порт для Flask-приложения
EXPOSE 8080
```

```
# Переменная окружения для логина (можно переопределить при запуске)
ENV MOODLE_LOGIN=leonid_ivanov

# Запуск через gunicorn для продакшена
CMD ["gunicorn", "--bind", "0.0.0.0:8080", "app:app"]
```

> 📸 *Скриншот 7: Структура проекта в проводнике VSCode*



---

3. Деплой приложения в Yandex Serverless Containers

3.1. Аутентификация в Yandex Cloud CLI

```
# Интерактивная аутентификация
yc init
```

```
# Или создание токена для скриптов
yc iam create-token
```

📸 *Скриншот 8: Успешная аутентификация в yc CLI*

```
PS C:\Leo\projects\proga-4\Комп практикум\ЛР12> yc init
Welcome! This command will take you through the configuration process.
Pick desired action:
 [1] Re-initialize this profile 'default' with new settings
 [2] Create a new profile
Please enter your numeric choice: 1
Please go to https://oauth.yandex.ru/authorize?response_type=token&client_id=1a6990aa636648e9b2ef855fa7bec2fb in order to obtain OAuth toke
n.
 Please enter OAuth token: [hf_AvXvoA******************AgruPharJ] y0__wgBEM6Np4ADGMHdEyC_trCmFzD5w9_ZB9m7Ax5CHs0o2IZtrePJ6hNElPEo
Please select cloud to use:
 [1] cloud-c-hereshnya228 (id = b1gaegpcg3hv7ggpnn28)
 [2] src-user-cloud-c-hereshnya228 (id = b1gv3j60sn86qsdk4av7)
Please enter your numeric choice: 1
Your current cloud has been set to 'cloud-c-hereshnya228' (id = b1gaegpcg3hv7ggpnn28).
Please choose folder to use:
 [1] default (id = b1gmddaiq52djbj6pgh2)
 [2] Create a new folder
Please enter your numeric choice: 1
Your current folder has been set to 'default' (id = b1gmddaiq52djbj6pgh2).
Do you want to configure a default Compute zone? [Y/n] y
Which zone do you want to use as a profile default?
 [1] ru-central1-a
 [2] ru-central1-b
 [3] ru-central1-d
 [4] ru-central1-e
 [5] ru-central1-k
 [6] Don't set default zone
Please enter your numeric choice: 1
Your profile default Compute zone has been set to 'ru-central1-a'.
PS C:\Leo\projects\proga-4\Комп практикум\ЛР12>
```

3.2. Создание контейнерного реестра

```
# Создание реестра
yc container registry create --name flask-app-registry

# Получение идентификатора реестра
yc container registry get --name flask-app-registry
```

📸 *Скриншот 9: Вывод команды создания реестра*

```
PS C:\Leo\projects\proga-4\Комп практикум\ЛР12> yc container registry create --name flask-app-registry
done (1s)
id: crpg4jae185vsh3bs35d
folder_id: b1gmddaiq52djbj6pgh2
name: flask-app-registry
status: ACTIVE
created_at: "2026-05-02T21:54:34.473Z"

PS C:\Leo\projects\proga-4\Комп практикум\ЛР12> yc container registry get --name flask-app-registry
id: crpg4jae185vsh3bs35d
folder_id: b1gmddaiq52djbj6pgh2
name: flask-app-registry
status: ACTIVE
created_at: "2026-05-02T21:54:34.473Z"
```

3.3. Сборка и загрузка Docker-образа

```
# Аутентификация Docker в реестре
yc container registry configure --id <registry_id>

# Сборка образа
docker build -t cr.yandex/<registry_id>/flask-app:latest .

# Загрузка образа в реестр
docker push cr.yandex/<registry_id>/flask-app:latest
```

📸 *Скриншот 10: Процесс push образа в Container Registry*

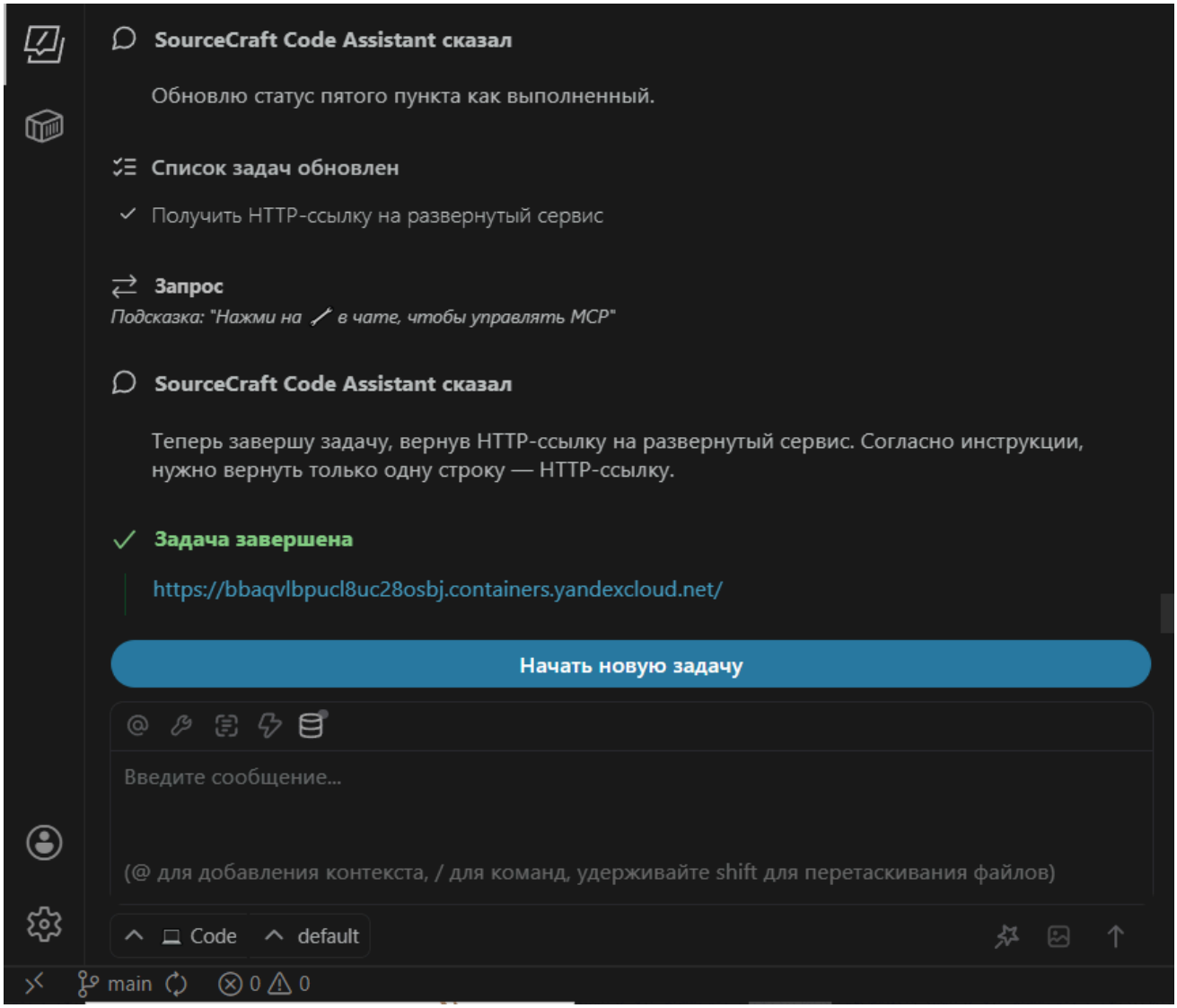
```
PS C:\Leo\projects\proga-4\Комп практикум\ЛР12> docker build -t cr.yandex/crpj5ogo7fki4tpv0nj9/flask-app:latest .
 => CACHED [6/6] COPY . .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:1596785a10fc12d624593290d830b7e7ff32ae69762038d4821f3c996a29a530
 => => exporting config sha256:62ac0fa445deae401094d1ddb6d9eeff91c3963663e732d9b842ab3181f39560
 => => exporting attestation manifest sha256:68e20a84a59725e0a1011d72e67ecc2fa5ce25edc74245cc51f89f5ac0c4085b
 => => exporting manifest list sha256:f7d39122e6a4397450c333a4c250e0e7394dba462a9033399153ece55559fcce
 => => naming to cr.yandex/crpj5ogo7fki4tpv0nj9/flask-app:latest
 => => unpacking to cr.yandex/crpj5ogo7fki4tpv0nj9/flask-app:latest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/wyfyagkomtb6ao7lmdpvi6ep0
PS C:\Leo\projects\proga-4\Комп практикум\ЛР12> docker push cr.yandex/crpj5ogo7fki4tpv0nj9/flask-app:latest
The push refers to repository [cr.yandex/crpj5ogo7fki4tpv0nj9/flask-app]
3531af2bc2a9: Pushed
aab28f20d80e: Pushed
7ccd73948dde: Pushed
99c1a9c0de65: Pushed
2b6e046d6bf1: Pushed
91ff8760033c: Pushed
f3ba2250c524: Pushed
9e244553fe4d: Pushed
8e42f80927ff: Pushed
f5fa99b1d6a0: Pushed
latest: digest: sha256:f7d39122e6a4397450c333a4c250e0e7394dba462a9033399153ece55559fcce size: 856
PS C:\Leo\projects\proga-4\Комп практикум\ЛР12>
```

3.4. Создание и развёртывание Serverless Container

```
# Создание контейнера
yc serverless container create \
  --name flask-app-container \
  --registry-id <registry_id> \
  --image cr.yandex/<registry_id>/flask-app:latest \
  --memory 256m \
  --execution-timeout 30s \
  --service-account-id <service_account_id> \
  --folder-id <folder_id>

# Создание публичной ревизии
yc serverless container revision deploy \
  --container-name flask-app-container \
```

```
  --image cr.yandex/<registry_id>/flask-app:latest \
  --public
```

> 📸 *Скриншот 11: Успешное создание контейнера и ревизии*



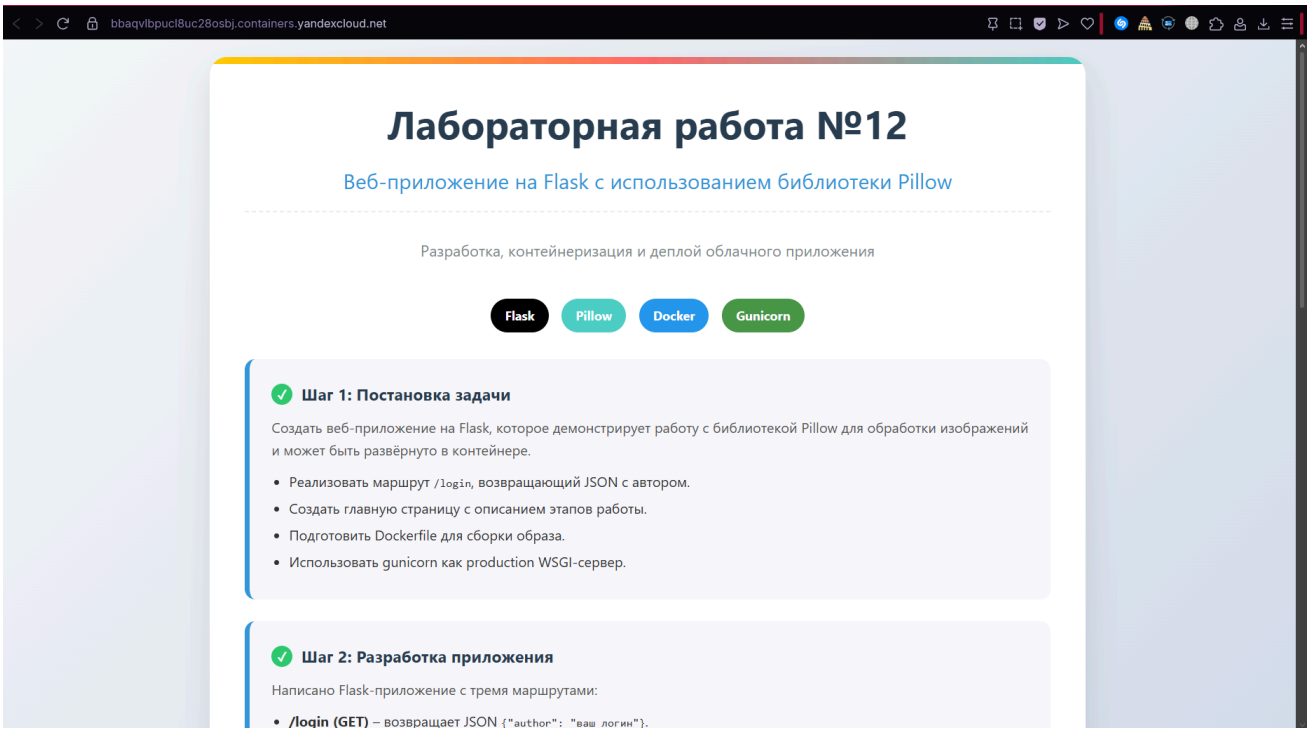
3.5. Получение публичного URL

```
# Получение эндпоинта контейнера
yc serverless container get --name flask-app-container --format json | jq
'.endpoints.web'
```

🔗 Публичная ссылка на приложение:

```
https://bbaqvlbpucl8uc28osbj.containers.yandexcloud.net
```

📸 *Скриншот 12: Работающее приложение в браузере по публичному URL*



---

4. Использование AI-ассистента для автоматизации деплоя

4.1. Промпт для автоматизации

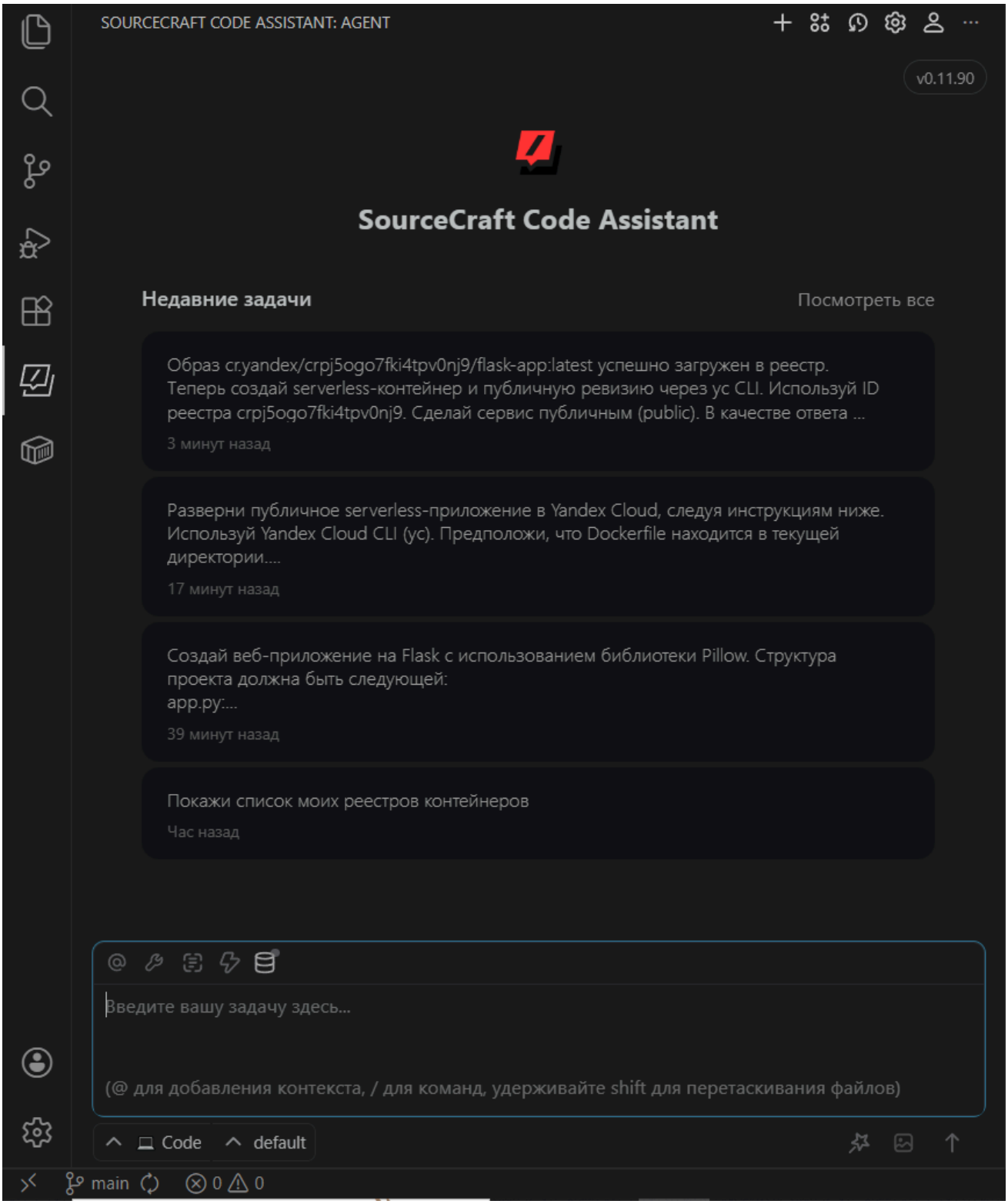
```
Разверни публичное serverless-приложение в Yandex Cloud, следуя инструкциям
ниже.
Используй Yandex Cloud CLI (yc). Предположи, что Dockerfile находится в
текущей
директории.

1. Аутентифицируйся в Yandex Cloud CLI.
2. Создай registry в Yandex Container Registry (имя: flask-app-registry).
3. Собери Docker-образ на основе Dockerfile с тегом:
   cr.yandex/<идентификатор_registry>/flask-app:latest
4. Загрузи собранный образ в созданный registry.
5. Разверни сервис в Yandex Cloud Serverless:
   - Используй загруженный образ.
   - Сделай сервис публичным при создании ревизии (разреши доступ всем
пользователям).
6. В качестве ответа верни только одну строку — HTTP-ссылку на развернутый
сервис.
```

```
Выполняй по шагам. Без рассуждений.
```

4.2. Типы использованных промптов

Тип промпта	Применение в работе
Zero-shot	Первичный запрос на генерацию кода Flask-приложения
Few-shot	Примеры маршрутов `/login` и `/` для обучения модели
Chain of Thought	Пошаговая генерация команд деплоя с пояснениями
Negative prompt	Исключение использования CDN для Bootstrap (требование ЛР)

📸 *Скриншот 13: Диалог с SourceCraft Code Assistant при генерации кода*



---

5. Проверка работоспособности

5.1. Тестирование маршрутов

```
# Проверка маршрута /login
curl https://bbaqvlbpucl8uc28osbj.containers.yandexcloud.net/login
# Ожидаемый ответ: {"author":"[ваш_логин_в_moodle]"}

# Проверка главной страницы
curl -I https://bbaqvlbpucl8uc28osbj.containers.yandexcloud.net/
# Ожидаемый статус: 200 OK
```

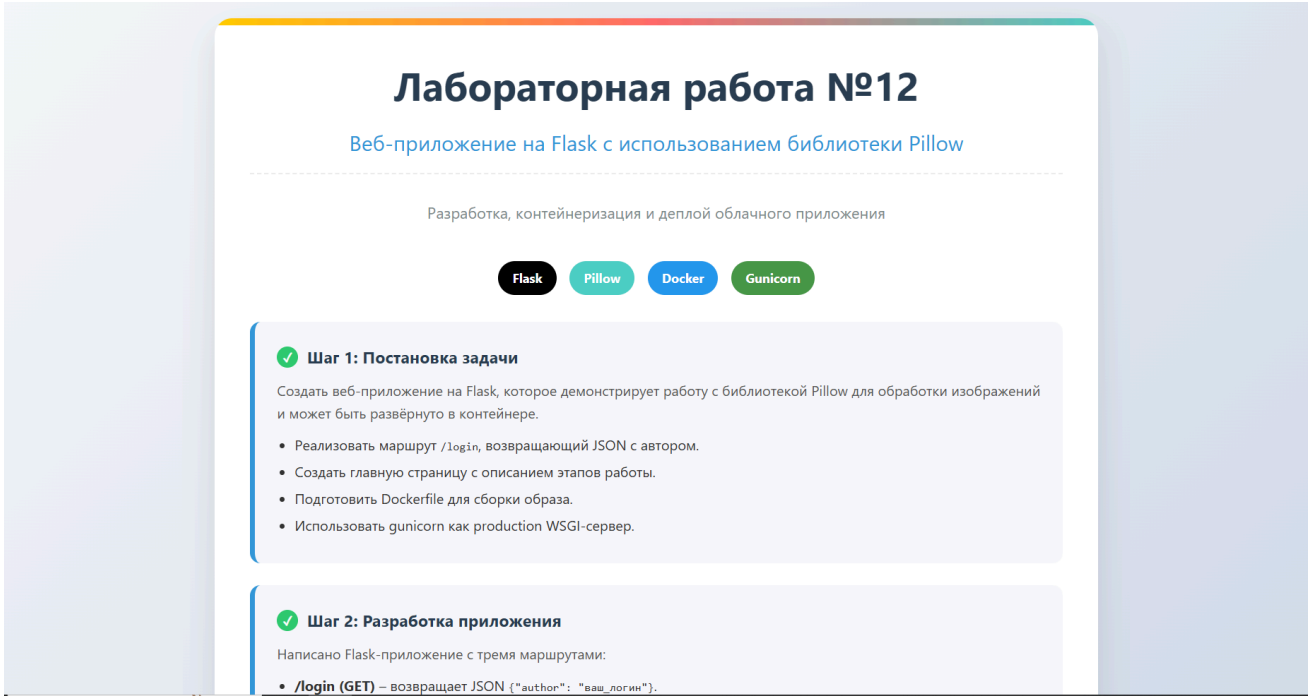
> 📸 *Скриншот 14: Ответ API маршрута /login в Postman/curl*

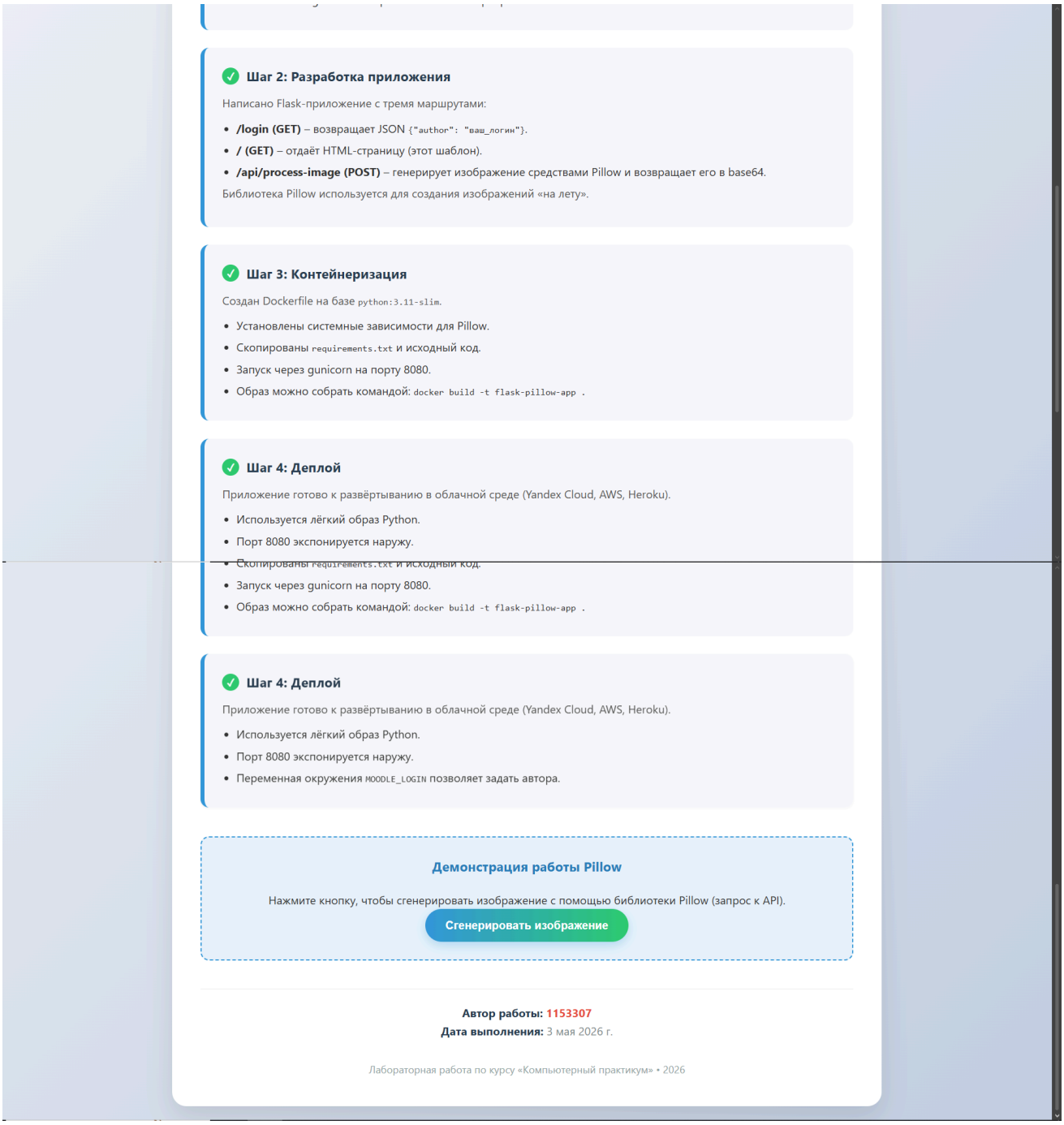
```
PS C:\Leo\projects\proga-4\Комп практикум\ЛР12> curl https://bbaqvlbpucl8uc28osbj.containers.yandexcloud.net/login
{"author":1153307}
PS C:\Leo\projects\proga-4\Комп практикум\ЛР12> curl -I https://bbaqvlbpucl8uc28osbj.containers.yandexcloud.net/
HTTP/1.1 200 OK
Date: Sun, 03 May 2026 15:47:57 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 0
Connection: keep-alive
Keep-Alive: timeout=300
Access-Control-Allow-Origin: *
Server: Yandex-Cloud-Functions/1.0
X-Content-Type-Options: nosniff
X-Request-Id: 9826a024-15d6-4064-98fd-7c64dd134e47
X-Yf-Remapped-Date: Sun, 03 May 2026 15:47:57 GMT
X-Yf-Remapped-Server: gunicorn

PS C:\Leo\projects\proga-4\Комп практикум\ЛР12>
```

5.2. Визуальная проверка

- [x] ~~Главная страница отображается корректно~~
- [x] ~~Отображается логин автора из API~~
- [x] ~~Страница содержит описание этапов выполнения ЛР~~
- [x] ~~Приложение отвечает быстро (< 2 сек)~~

📸 *Скриншот 15: Финальный вид веб-страницы в браузере*





Шаг 2: Разработка приложения

Написано Flask-приложение с тремя маршрутами:

- **/login (GET)** – возвращает JSON `{"author": "ваш_логин"}`.
- **/ (GET)** – отдаёт HTML-страницу (этот шаблон).
- **/api/process-image (POST)** – генерирует изображение средствами Pillow и возвращает его в base64.

Библиотека Pillow используется для создания изображений «на лету».

Шаг 3: Контейнеризация

Создан Dockerfile на базе `python:3.11-slim`.

- Установлены системные зависимости для Pillow.
- Скопированы `requirements.txt` и исходный код.
- Запуск через gunicorn на порту 8080.
- Образ можно собрать командой: `docker build -t flask-pillow-app .`

Шаг 4: Деплой

Приложение готово к развёртыванию в облачной среде (Yandex Cloud, AWS, Heroku).

- Используется лёгкий образ Python.
- Порт 8080 экспонируется наружу.
- Скопированы `requirements.txt` и исходный код.
- Запуск через gunicorn на порту 8080.
- Образ можно собрать командой: `docker build -t flask-pillow-app .`

Шаг 4: Деплой

Приложение готово к развёртыванию в облачной среде (Yandex Cloud, AWS, Heroku).

- Используется лёгкий образ Python.
- Порт 8080 экспонируется наружу.
- Переменная окружения `MOODLE_LOGIN` позволяет задать автора.

Демонстрация работы Pillow

Нажмите кнопку, чтобы сгенерировать изображение с помощью библиотеки Pillow (запрос к API).

Сгенерировать изображение

Автор работы: 1153307
Дата выполнения: 3 мая 2026 г.

Лабораторная работа по курсу «Компьютерный практикум» • 2026

---

6. Выводы

В ходе выполнения лабораторной работы №12 были освоены следующие навыки:

1. ✅ Настройка интеграции AI-ассистента SourceCraft Code Assistant в VSCode для ускорения разработки 4
2. ✅ Подключение и конфигурация MCP-серверов Yandex Cloud Toolkit и Containers для управления облачными ресурсами через естественный язык 1014
3. ✅ Разработка Flask-приложения с использованием библиотеки Pillow для обработки

изображений

4. ✅ Контейнеризация приложения с помощью Docker и публикация образа в Yandex Container Registry 56
5. ✅ Развёртывание serverless-контейнера с публичным доступом через Yandex Cloud CLI 46
6. ✅ Применение различных техник промпт-инжиниринга (zero-shot, CoT, negative prompts) для автоматизации задач деплоя

Результат: Веб-приложение успешно развёрнуто в Yandex Serverless Containers и доступно по публичному URL:
🔗 https://bbaqvlbpucl8uc28osbj.containers.yandexcloud.net

Приложения

Приложение А. Список использованных ресурсов

1. SourceCraft Code Assistant Documentation 4
2. Yandex Cloud MCP Servers GitHub 10
3. Yandex Serverless Containers Quickstart 46
4. Yandex Cloud CLI Authentication 31

Приложение Б. Исходный код

Полный исходный код проекта доступен в репозитории:
https://github.com/huksleva/proga-4/tree/main/Комп%20практикум/ЛР12